

# Reviewer Pack — Greedy Bounded-Asdim Proxy $\hat{\kappa}_R$ Is $O(\log n)$ -Lipschitz Under ...

Entry 8a527737c659 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 11:14:58 UTC

## Greedy Bounded-Asdim Proxy $\hat{\kappa}_R$ Is $O(\log n)$ -Lipschitz Under Restriction

Entry ID: 8a527737c659 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 11:14:58 UTC

### 1. Conjecture

**Field A** (mathematical branch): Asymptotic dimension (Hamming-cube bounded-asdim cover proxy)

**Field B** (complexity object): Boolean function complexity (single-variable restrictions of Boolean functions)

#### Statement:

For  $f: \{0,1\}^n \rightarrow \{0,1\}$ , let  $S(f)$  be a fixed seed-deterministic sample of  $m=64$  satisfying assignments (full set when  $|f^{-1}(1)| \leq 64$ ). Define the greedy bounded-asdim proxy  $\hat{\kappa}_R(f)$  by the procedure: while  $S(f)$  is uncovered, pick  $c_j \in S(f)$  maximizing the number of uncovered points within Hamming distance  $R$  (break ties by smallest integer encoding), mark all such points covered; then set  $\hat{\kappa}_R(f) := (\max\{y \in S(f) \mid \#\{j : \text{Hamming}(c_j, y) \leq R + \lfloor R/4 \rfloor\} = y\}) - 1$ . Conjecture: for every  $f$  on  $n \leq 40$ , every  $i \in [n]$ , every  $b \in \{0,1\}$ , and every  $R$  with  $1 \leq R \leq n/2$ ,  $|\hat{\kappa}_R(f|_{x_i=b}) - \hat{\kappa}_R(f)| \leq 2 \cdot \lceil \log_2(n+1) \rceil$ .

#### Rationale (proposer's reasoning):

For  $\kappa$  to serve as the computable definition installed in CGKW.lean per SC1, it must satisfy a discrete Lipschitz-under-restriction axiom — restricting one Boolean coordinate is a coarse-geometric quasi-isometric embedding at unit scale and asdim is monotone under coarse embeddings. The greedy radius- $R$  ball cover is the canonical polynomial-time witness of bounded asymptotic dimension on a discrete metric, so its restriction-Lipschitz constant is exactly the quantity SC1 needs to pin down before  $\kappa$  can be used. A logarithmic Lipschitz bound would certify  $\hat{\kappa}$  as the installable proxy and unblock the next-step verification of axiom A1 for Andreev's function.

**Taxonomy category:** cg\_kw\_andreev (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** d5cf987bec94ffe8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion:**

Across 180 trials (30 seeds  $\times$  6 values of  $n \in \{6, 10, 16, 24, 32, 40\}$ ), record  $\delta := |\hat{\kappa}_R(f|_{\{x_i=b\}}) - \hat{\kappa}_R(f)|$  and threshold  $T(n) := 2 \cdot \lceil \log_2(n+1) \rceil$ . Conjecture SUPPORTED iff every trial satisfies  $\delta \leq T(n)$ ; FALSIFIED iff any single trial has  $\delta > T(n)$ .

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.95       | SAFE             | SAFE             |
| NATURAL_PROOFS | SAFE    | 0.95       | SAFE             | SAFE             |
| ALGEBRIZATION  | SAFE    | 0.92       | SAFE             | SAFE             |
| KARP_LIPTON    | SAFE    | 0.99       | SAFE             | SAFE             |

### 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - asymptotic dimension Hamming cube Boolean function restriction Lipschitz-greedy cover multiplicity proxy bounded asymptotic dimension Boolean-Assouad-Nagata dimension Hamming metric Boolean function sensitivity restriction

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=124, elapsed=240.0s

#### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def hamming_distance(x, y):
        return sum(xi != yi for xi, yi in zip(x, y))

    def greedy_ball_cover(f, S, R):
        covered = [False] * len(S)
        count = 0
        while not all(covered):
            max_points = []
            max_count = 0
            for c_j in S:
                if not covered[S.index(c_j)]:
                    points_in_range = [S[i] for i in range(len(S)) if hamming_distance(c_j, S[i]) < R]
                    if len(points_in_range) > max_count:
                        max_points = points_in_range
                        max_count = len(points_in_range)
            if not max_points:
                break
            count += 1
            for p in max_points:
```

```

        covered[S.index(p)] = True
    return count

def sample_satisfying_assignments(f, m):
    S = []
    for _ in range(m):
        while True:
            assignment = [random.choice([0, 1]) for _ in range(n)]
            if f(assignment) == 1:
                S.append(assignment)
                break
    return S

n_values = [6, 10, 16, 24, 32, 40]
results = []

for n in n_values:
    m = 64
    f = lambda x: random.choice([0, 1]) if n <= 12 else any(random.choice([True, False]) for _ in range(n))
    S = sample_satisfying_assignments(f, m)

    for R in range(1, n // 2 + 1):
        i = random.randint(0, n - 1)
        b = random.choice([0, 1])

        f_prime = lambda x: f(x[:i] + [b] + x[i+1:])
        delta = abs(greedy_ball_cover(f, S, R) - greedy_ball_cover(f_prime, S, R))
        results.append({
            "metric_name": "delta",
            "metric_value": delta,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": delta <= 2 * math.ceil(math.log2(n + 1)),
            "counterexample": "" if delta <= 2 * math.ceil(math.log2(n + 1)) else f"delta={delta}"
        })

    return {
        "seed": seed,
        "metric_name": "delta",
        "metric_value": sum(result["metric_value"] for result in results) / len(results),
        "instances_tested": len(results),
        "n_max": max(result["n_max"] for result in results),
        "conjecture_holds": all(result["conjecture_holds"] for result in results),
        "counterexample": next((result["counterexample"] for result in results if not result["conjecture_holds"]), "")
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    results = [run_trial(seed) for seed in seeds]
    mean_delta = sum(result["metric_value"] for result in results) / len(results)

```

```

support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_delta} std=0.0 support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

TIMEOUT after 240s

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test timed out after 240s before producing any RESULT line or trial data, so the pre-registered support condition cannot be evaluated. | next: Optimize the greedy covering procedure (e.g., precompute Hamming-ball memberships or reduce m/R search) and re-run with a longer timeout or smaller n grid.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | claude_max    | opus             | 0         | 0   | 93043        |
| 2 | preregistration | claude_max    | opus             | 0         | 0   | 5493         |
| 3 | novelty         | claude_max    | opus             | 0         | 0   | 3220         |
| 4 | novelty         | claude_max    | opus             | 0         | 0   | 5924         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 16449        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15395        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17517        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15370        |
| 9 | judge           | claude_max    | opus             | 0         | 0   | 3766         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 176179 ms total latency. Provider mix: {'claude\_max': 5, 'ollama\_remote': 4}

(full prompt+response transcripts available in [research/audit/8a527737c659.jsonl](#))

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8a527737c659.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/8a527737c659.tar.gz` (if generated)